

# DSA - Seminar 5

---

1. Evaluate an arithmetic expression which contains single digit operands, parantheses and the +, -, \*, / operators. **We assume that the expression is correct.** For example:
  - $2+3*4 = 14$
  - $((2+4)*7)+3*(9-5) = 54$
  - $2*(4+3)-4+6/2*(1+2*7)+4 = 59$

The expressions from the above example are in the so called *infix* notation. This means that every operator is between the two operands it refers to (for example:  $2+4$ ). This is how we work with arithmetic expressions in general.

For a computer it is a lot easier to work with the *postfix* notation. This is a notation in which the operator comes after the two operands (for example:  $2\ 4\ +$ ).

A few examples of expressions in the infix notation and their corresponding postfix notation:

|                     |                       |
|---------------------|-----------------------|
| $2 + 4$             | $2\ 4\ +$             |
| $4*3+6$             | $4\ 3\ *\ 6\ +$       |
| $4*(3+6)$           | $4\ 3\ 6\ +\ *$       |
| $(5 + 6) * (4 - 1)$ | $5\ 6\ +\ 4\ 1\ -\ *$ |

Obs:

- Relative order of the operands stays the same
- Relative order of the operators might change.
- We no longer have parentheses in the postfix notation

Thus, evaluating an arithmetic expression will have two steps:

- Transform the expression in the corresponding postfix notation
- Evaluate the expression in the postfix notation

1. Transform an infix expression in the corresponding postfix notation. Input is the infix expression, output will be the postfix notation in the form of a queue and for the transformation we use an auxiliary stack.

Steps:

- Start parsing the expression. For every element, you will have one of the following cases:

**Case 1.** Current element is an operand => push it to the queue

**Case 2.** Current element is an open parenthesis => push it to the stack

**Case 3.** Current element is a closed parenthesis => open parenthesis should be on the stack already, so pop everything from the stack (and whenever you pop something, push it to the queue) until you get to the first open parenthesis. Do not push the open or closed parenthesis to the queue, but pop the open one from the stack.

**Case 4.** If you find an operator:

- As long as the stack is not empty and the top element of the stack is an operator with a priority greater than or equal to the priority of the current operator, pop from stack and push to the queue.
- Push operator to the stack

**Case 5.** There are no more current elements (expression is over) => pop whatever you have left on the stack and push it to the queue.

Example:  $2*(4+3)-4+6/2*(1+2*7)+9/(1+1*4/2)+6+2*8-4*(8/(2+6-4)+1)+5$

| Current element | Case                                                                                | Stack (top is on the right) | Queue                 |
|-----------------|-------------------------------------------------------------------------------------|-----------------------------|-----------------------|
| 2               | 1. Push to queue                                                                    |                             | 2                     |
| *               | 4. Push to stack                                                                    | *                           |                       |
| (               | 2. Push to stack                                                                    | * (                         |                       |
| 4               | 1. Push to queue                                                                    |                             | 2 4                   |
| +               | 4. Push to stack                                                                    | * ( +                       |                       |
| 3               | 1. Push to queue                                                                    |                             | 2 4 3                 |
| )               | 3. Pop from stack and push to queue until you find the open parenthesis             | *                           | 2 4 3 +               |
| -               | 4. Pop from stack as long as top has greater or equal priority. Push - to the stack | -                           | 2 4 3 + *             |
| 4               | 1. Push to queue                                                                    |                             | 2 4 3 + * 4           |
| +               | 4. Pop from stack as long as top has greater or equal priority. Push + to stack     | +                           | 2 4 3 + * 4 -         |
| 6               | 1. Push to queue                                                                    |                             | 2 4 3 + * 4 - 6       |
| /               | 4. Push to stack                                                                    | + /                         |                       |
| 2               | 1. Push to queue                                                                    |                             | 2 4 3 + * 4 - 6 2     |
| *               | 4. Pop from stack as long as top has greater or equal priority. Push * to stack     | + *                         | 2 4 3 + * 4 - 6 2 /   |
| (               | 2. Push to stack                                                                    | + * (                       |                       |
| 1               | 1. Push to queue                                                                    |                             | 2 4 3 + * 4 - 6 2 / 1 |
| +               | 4. Push to stack                                                                    | + * ( +                     |                       |

|   |                                                                                     |             |                                                                                      |
|---|-------------------------------------------------------------------------------------|-------------|--------------------------------------------------------------------------------------|
| 2 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2$                                                        |
| * | 4. Push to stack                                                                    | $+ * ( + *$ |                                                                                      |
| 7 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7$                                                     |
| ) | 3. Pop from stack and push to queue until you find the open parenthesis             | $+ *$       | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * +$                                                 |
| + | 4. Pop from stack as long as top has greater or equal priority. Push + to the stack | $+$         | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * +$                                             |
| 9 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9$                                           |
| / | 4. Push to stack                                                                    | $+ /$       |                                                                                      |
| ( | 2. Push to stack                                                                    | $+ / ($     |                                                                                      |
| 1 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1$                                        |
| + | 4. Push to stack                                                                    | $+ / ( +$   |                                                                                      |
| 1 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1$                                     |
| * | 4. Push to stack                                                                    | $+ / ( + *$ |                                                                                      |
| 4 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4$                                  |
| / | 4. Pop from stack * and push to queue and push / to stack                           | $+ / ( + /$ | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 *$                                |
| 2 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2$                              |
| ) | 3. Pop from stack and push to queue until open parenthesis                          | $+ /$       | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / +$                          |
| + | 4. Pop / and + from stack and push to queue, push +                                 | $+$         | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / +$                      |
| 6 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6$                    |
| + | 4. Pop + from stack and push to queue, push +                                       | $+$         | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 +$                  |
| 2 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 + 2$                |
| * | 4. Push to stack                                                                    | $+ *$       |                                                                                      |
| 8 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 + 2\ 8$             |
| - | 4. Pop * and + from stack, push to queue, push - to stack                           | $-$         | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 + 2\ 8 * +$         |
| 4 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 + 2\ 8 * + 4$       |
| * | 4. Push to stack                                                                    | $- *$       |                                                                                      |
| ( | 2. Push to stack                                                                    | $- * ($     |                                                                                      |
| 8 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 + 2\ 8 * + 4\ 8$    |
| / | 4. Push to stack                                                                    | $- * ( /$   |                                                                                      |
| ( | 2. Push to stack                                                                    | $- * ( ($   |                                                                                      |
| 2 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4 * 2 / + / + 6 + 2\ 8 * + 4\ 8\ 2$ |
| + | 4. Push to stack                                                                    | $- * ( ( +$ |                                                                                      |
| 6 | 1. Push to queue                                                                    |             | $2\ 4\ 3 + * 4 - 6\ 2 / 1\ 2\ 7 * + * + 9\ 1\ 1\ 4$                                  |

|                    |                                                        |             |                                                                                                         |
|--------------------|--------------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------|
|                    |                                                        |             | $* 2 / + / + 6 + 2 8 * + 4 8 2 6$                                                                       |
| -                  | 4. Pop – from stack and push to queue, push + to stack | - * ( / ( - | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 +$                      |
| 4                  | 1. Push to queue                                       |             | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4$                    |
| )                  | 3. Pop – and ( from stack, push – to queue             | - * ( /     | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 -$                  |
| +                  | 4. Pop / from stack and push to queue, push + to stack | - * ( +     | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 - /$                |
| 1                  | Push to queue                                          |             | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 - / 1$              |
| )                  | 3. Pop + and ) and push to queue                       | - *         | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 - / 1$<br>+         |
| +                  | 4. Pop * and – and push to queue, push + to stack      | +           | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 - / 1$<br>+ * -     |
| 5                  | 1. Push to queue                                       |             | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 - / 1$<br>+ * - 5   |
| Expression is over | 5. Pop all from stack and push to queue                |             | $2 4 3 + * 4 - 6 2 / 1 2 7 * + * + 9 1 1 4$<br>$* 2 / + / + 6 + 2 8 * + 4 8 2 6 + 4 - / 1$<br>+ * - 5 + |

Pseudocode implementation:

Assume stack and queue is already implemented and they have the standard operations: init, push, pop, top, isEmpty.

**function** transform(expression) **is**:

  //create stack and queue

  init(st)

  init(q)

**for** e **in** expression **execute**:

**if** e **is** operand **then**

    push(q, e)

**else if** e **is** '(' **then**

    push(st, e)

**else if** e **is** ')' **then**

**while** top(st) != '(' **execute**

      elem <- pop(st)

      push(q, elem)

**end-while**

    pop(st) //to remove the (

**else** //e **is** operand

**while** (not isEmpty(st)) **AND** (top(st) != '(') **AND**

      (top(st) has higher or equal priority than e) **execute**

      elem <- pop(st)

      push(q, elem)

```

        end-while
        push(st, e)
    end-if
end-while
while not isEmpty(st) execute:
    elem <- pop(st)
    push(q, elem)
end-while
transform <- q
end-function

```

2. Evaluate the postfix expression. Input is the postfix notation in the form of a queue (actually the output of step 1) and the output will be a value: the result of the evaluation. In the process we use an auxiliary stack.
  - Start parsing the expression from the queue.

**Case 1:** current element is an operand => push to the stack

**Case 2:** current element is an operator (this is postfix notation, operators are after operands, so the operands have to be in the stack already) => pop two elements from the stack (these are the operands for this operator), perform the operation and push the result back to the stack

**Case 3:** When the queue is empty, the stack contains one single element, this is the result

| Current elem from queue | What to do                                      | Stack (top is on the right) |
|-------------------------|-------------------------------------------------|-----------------------------|
| 2                       | Push to stack                                   | 2                           |
| 4                       | Push to stack                                   | 2 4                         |
| 3                       | Push to stack                                   | 2 4 3                       |
| +                       | Pop 2 elements, perform +, push result to stack | 2 7                         |
| *                       | Pop 2 elements, perform *, push result to stack | 14                          |
| 4                       | Push to stack                                   | 14 4                        |
| -                       | Pop 2 elements, perform -, push result to stack | 10                          |
| 6                       | Push to stack                                   | 10 6                        |
| 2                       | Push to stack                                   | 10 6 2                      |
| /                       | Pop 2 elements, perform /, push result to stack | 10 3                        |
| 1                       | Push to stack                                   | 10 3 1                      |
| 2                       | Push to stack                                   | 10 3 1 2                    |
| 7                       | Push to stack                                   | 10 3 1 2 7                  |
| *                       | Pop 2 elements, perform *, push result to stack | 10 3 1 14                   |
| +                       | Pop 2 elements, perform +, push result to stack | 10 3 15                     |
| *                       | Pop 2 elements, perform *, push result to stack | 10 45                       |
| +                       | Pop 2 elements, perform +, push result to stack | 55                          |
| 9                       | Push to stack                                   | 55 9                        |
| 1                       | Push to stack                                   | 55 9 1                      |
| 1                       | Push to stack                                   | 55 9 1 1                    |
| 4                       | Push to stack                                   | 55 9 1 1 4                  |

|                    |                                                     |            |
|--------------------|-----------------------------------------------------|------------|
| *                  | Pop 2 elements, perform *, push result to stack     | 55 9 1 4   |
| 2                  | Push to stack                                       | 55 9 1 4 2 |
| /                  | Pop 2 elements, perform /, push the result to stack | 55 9 1 2   |
| +                  | Pop 2 elements, perform +, push the result to stack | 55 9 3     |
| /                  | Pop 2 elements, perform /, push te results to stack | 55 3       |
| +                  | Pop 2 elements, perform +, push result to stack     | 58         |
| 6                  | Push to stack                                       | 58 6       |
| +                  | Pop 2 elements, perform +, push the result to stack | 64         |
| 2                  | Push to stack                                       | 64 2       |
| 8                  | Push to stack                                       | 64 2 8     |
| *                  | Pop 2 elements, perform *, push the result to stack | 64 16      |
| +                  | Pop 2 elements, perform +, push the result to stack | 80         |
| 4                  | Push to stack                                       | 80 4       |
| 8                  | Push to stack                                       | 80 4 8     |
| 2                  | Push to stack                                       | 80 4 8 2   |
| 6                  | Push to stack                                       | 80 4 8 2 6 |
| +                  | Pop 2 elements, perform +, push the result to stack | 80 4 8 8   |
| 4                  | Push to stack                                       | 80 4 8 8 4 |
| -                  | Pop 2 elements, perform -, push the result to stack | 80 4 8 4   |
| /                  | Pop 2 elements, perform /, push the result to stack | 80 4 2     |
| 1                  | Push to stack                                       | 80 4 2 1   |
| +                  | Pop 2 elements, perform +, push the result to stack | 80 4 3     |
| *                  | Pop 2 elements, perform *, push the result to stack | 80 12      |
| -                  | Pop 2 elements, perform -, push the result to stack | 68         |
| 5                  | Push to stack                                       | 68 5       |
| +                  | Pop 2 elements, perform +, push the result to stack | 73         |
| Expression is over | Pop element from stack, it is the result: 73        |            |
|                    |                                                     |            |

Pseudocode implementation

```

function evaluate(postfix) is:
  init(st)
  while not isEmpty(postfix) execute:
    elem <- pop(postfix)
    if elem is operand then
      push (st, elem)
    else
      op1 <- pop(st)
      op2 <- pop(st)
      res<- @compute the result of op2 elem op1
      push(st, res)
    end-if
  end-while
  result <- pop(st)
  evaluate <- result
end-function

```

2. Consider the following problem: Determine the sum of the largest  $k$  elements from a vector containing  $n$  distinct numbers. For example, if the array contains the following 10 elements [6, 12, 9, 91, 3, 5, 25, 81, 11, 23] and  $k = 3$ , the result should be:  $91 + 81 + 25 = 197$ .
  - I. Find the maximum  $k$  times (especially good if  $k$  is small)
    - If we just call the maximum function 3 times for our example, it will return 91 each time, so we need a solution where we also have an upper bound, and we are searching for the maximum which is less than that value.
    - First, maximum is 91. At the second call we want the maximum which is less than 91, we will get 81.
    - At the third call we want the maximum which is less than 81, we will get 25.
    - Complexity of the approach:  $\Theta(k*n)$  – finding the maximum is  $\Theta(n)$  and we do this  $k$  times.
  - II. Similarly, we could do just  $k$  iterations of some sorting algorithms:
    - Selection sort (with descending sorting) finds the maximum and moves it to position 1. Then it finds the maximum of the remaining array and moves it to the second positions, etc. After  $k$  iterations the first  $k$  elements will be the ones we need to add together.
    - Bubble sort (with ascending sorting) – one iteration of bubble sort will move the maximum element to the end of the array. The next iteration will guarantee that the element on position  $n-1$  is the second largest, etc. After  $k$  iterations, the last  $k$  elements will be the ones we need to sum up.
    - Complexity for both cases is  $\Theta(k*n)$
  - III. Sort the array in a descending order and pick the first  $k$  elements (especially good if  $k$  is large).
    - Sorting can be done in  $\Theta(n*\log_2 n)$  time
    - Computing the sum of the first  $k$  elements:  $\Theta(k)$
    - In total  $\Theta(n*\log_2 n) + \Theta(k) \in \Theta(n*\log_2 n)$
  - IV. Keep a sorted array of the  $k$  largest elements found so far. Initially this array will contain the first  $k$  elements (in sorted order). For the next elements of the input array, we compare the element with the minimum of the  $k$  selected numbers and if necessary remove the minimum and add the new element. For our example:
    - Initially the array contains [12, 9, 6] (I chose to keep them in descending order, but it would be the same with ascending ordering).
    - We process 91, it is greater than 6 (the minimum of the selected  $k$  elements), so we remove 6 and add 91. Our array of selected elements will be: [91, 12, 9]
    - We process 3, it is less than 9 (the minimum of the selected  $k$  elements), so we do not modify the array
    - We process 5, it is less than 9 (the minimum of the selected  $k$  elements), so we do not modify the array

- We process 25, it is greater than 9 (the minimum of the selected k elements) so we remove 9 and add 25. Our array of selected elements will be: [91, 25, 12].
- Etc.
- At the end, we need to compute the sum of the elements in k (or we can keep a variable that gets updated while we process the elements).
- Complexity:
  - i. Processing one element can have a best case (element is less than the minimum, we just do a comparison:  $\Theta(1)$ ) or a worst case (element is a new maximum, we need to move every element in the array one position to the right:  $\Theta(k)$ )  $\Rightarrow O(k)$ .
  - ii. We need to process every element  $\Rightarrow O(n*k)$

V. Similar to the previous solution, but keep an unsorted array of k elements. When a new element is processed, we need to find the minimum of the array, compare it with the element and, if the new element is greater, replace the minimum with it. For our example:

- Initially the array contains [6, 12, 9]
- We process 91, it is greater than 6 (the minimum of the selected k elements), so we replace 6 with 91. Our array of selected elements will be: [91, 12, 9]
- We process 3, it is less than 9 (the minimum of the selected k elements), so we do not modify the array
- We process 5, it is less than 9 (the minimum of the selected k elements), so we do not modify the array
- We process 25, it is greater than 9 (the minimum of the selected k elements) so we replace 9 with 25. Our array of selected elements will be: [91, 12, 25].
- Etc.
- At the end, we need to compute the sum of the elements in k (or we can keep a variable that gets updated while we process the elements).

Replacing is  $\Theta(1)$ , but finding the minimum is  $\Theta(k)$ . It can become  $O(k)$  if we keep the minimum in a variable, and we do not recompute when we did not change the array at the previous iteration. Since we need to process every element from the input array  $\Rightarrow O(n*k)$

- VI. Use a binary max-heap. Add all the elements to the heap and remove the first k.
- Adding an element to a heap with n elements is  $O(\log_2 n)$ .
  - Removing an element from a heap with n elements is  $\Theta(\log_2 n)$  (we only have best case – and  $O(\log_2 n)$  complexity when we can have equal elements).
  - In total we have  $O(n*\log_2 n) + O(k*\log_2 n)$ . Since  $n \geq k$ , this is  $O(n*\log_2 n)$

```
function sumOfK(elems, n, k) is:
//elems is an array of unique integer numbers
//n is the number of elements from elems
//k is the number of elements we want to sum up. Assume k <= n
```



```

init(h, "≥") //assume we have the Heap data structure implemented. We
initialize a heap with the relation "≥" (a max-heap)
for i ← 1, n execute
    add(h, elems[i]) //add operation was discussed at Lecture 8
end-for
sum ← 0
for i ← 1, k execute
    elem ← remove(h) //remove operation was discussed at Lecture 8
    sum ← sum + elem
end-for
sumOfK ← sum
end-function

```

VII. How can we reduce the complexity? Well, we do not need all the elements in the heap, we are always interested in the k largest ones (similar to solution V, but instead of sorted array of k elements, we will have heap of k elements). If we consider the example from above, we can work in the following way, always keeping just the k maximum elements up until now:

- Initially we keep 6, 12, 9
- When we get to 91, we can drop 6, because we know for sure that it is not going to be part of the 3 maximum numbers (we already have 3 numbers greater than this). So we keep 12, 91, 9.
- When we get to 3, we know it is not going to be part of the 3 maximum elements (We already have 3 elements greater than that). Similar with 5.
- When we get to 25, we can drop 9, and go on with 12, 91, 25.
- Etc.

We can keep the k elements that we consider in a heap. Should it be a min-heap or max-heap?

When we have the k largest elements at a given point, we will be interested in the minimum of these elements (this is what we compare to the new element that is considered and this is what we remove if we find a larger one) so it should be a min-heap.

```

function sumOfK2(elems, n, k) is:
//elems is an array of unique integer numbers
//n is the number of elements from elems
//k is the number of elements we want to sum up. Assume k ≤ n
init(h, "≤") //assume we have the Heap data structure implemented. We
initialize a heap with the relation "≤" (a min-heap)
for i ← 1, k execute //the first k elements are added "by default"
    add(h, elems[i])
end-for
for i ← k+1, n execute
    if elems[i] > getFirst(h) then //getFirst is an operation which returns
the first element from the heap.
        remove(h) //it returns the removed element, but we do not need it
        add(h, elems[i])
    end-if
sum ← 0
for i ← 1, k execute
    elem ← remove(h) //remove operation was discussed at Lecture 8
    sum ← sum + elem
end-for

```

```

sumOfK2 ← sum
end-function

```

- Complexity? Our heap has maximum  $k$  elements, so operations have a complexity of  $O(\log_2 k)$ . We call add at most  $n$  times (worst case, when every element is greater than the root of the heap) and remove at most  $n$  times. So in total we have  $O(n \cdot \log_2 k)$
  - If you do not use an already implemented heap, but have access to the representation, you can make the previous implementation slightly more efficient (complexity will not change though):
    - the last *for* will not have to remove elements, just simply to add up the sum of the elements from the heap array (but for this you need access to the array) – You can eliminate that *for* even if you don't have access to the representation, if you keep a variable with the sum so far: whenever you add to the heap, you add to the sum, whenever you remove from the heap you subtract from the sum.
    - the middle *for* loop will not have to do a remove and an add. You can just overwrite the element from position 1 (this is what would be removed anyway) with the newly added element and do a bubble-down on it.
- VIII. Can we improve complexity even more? We know that if we have an array we can transform it into a heap in a complexity  $O(n)$  – it was discussed at heapsort. So we can do something similar to version V, but instead of adding the elements one by one to the heap we could transform the array into a max-heap and then remove  $k$  elements from it. This approach has a complexity of  $O(n + k \cdot \log_2 n)$
- IX. Assuming positive numbers, depending on the interval from which we have the elements, we can do the following:
- Take an auxiliary boolean array of size equal to the maximum element (91 in our example).
  - Traverse the input array and set to TRUE the value in the auxiliary array on the positions equal to the elements in the array (so, positions 3, 5, 6, 9, 11, 12, 23, 25, 81, 91 will have the value TRUE)
  - Starting from the maximum value (91) traverse the auxiliary array towards the beginning and add the first  $k$  positions with the value TRUE.
  - Setting the values to TRUE for the input array elements is  $\Theta(n)$ , while finding the first (considered from the end of the array)  $k$  TRUE values has complexity  $O(\max\_value)$ . Since elements are distinct (and for this case they need to be positive),  $\max\_value \geq n$ , so complexity is  $O(\max\_value)$ , with a best case of  $\Theta(k)$  if the first  $k$  positions checked all have the value TRUE.
- X. Another approach is to do the following:
- Write a function, let's call it *countGreaterThanOrEqual*(*elems*, *n*, *threshold*) which counts how many elements we have in the *elems* array, which are greater than or equal to the value *threshold*.
  - Assume that we call *countGreaterThanOrEqual* with a threshold *t*. If the function returns the value *k*, we just need to sum up every element from the array which is

greater than or equal to  $t$ , and we have our solution. So, we should look for this threshold  $t$ .

- If the function returns a value larger than  $k$ , we should try a higher threshold (in order to reduce the number of returned values). If the function returns a value lower than  $k$ , we should try a lower threshold (to increase the number of returned elements).
- The threshold should be somewhere between the minimum and maximum values of the array. We can do a binary search on this interval to find the optimal threshold. Start with the middle value. If *countGreaterThanOrEqualTo* returns a value less than  $k$ , we need to reduce the threshold by making the maximum limit equal to the middle value minus 1. Alternatively, if the number returned is greater than  $k$ , we increase the threshold, by making the minimum limit equal to the middle value + 1.
- For example, in our case:
  - i. Initially minimum is 3, maximum is 91, middle is 47.
  - ii. *countGreaterThanOrEqualTo*(*elems*, *n*, 47) will return 2, which is not enough, we should reduce the threshold.
  - iii. Minimum will be 3, maximum 46, middle is going to be 24.
  - iv. *countGreaterThanOrEqualTo*(*elems*, *n*, 24) returns 3, which is equal to  $k$ . So we just need to sum up all values greater than 24 and we have our solution.
- *countGreaterThanOrEqualTo*, finding minimum and maximum and computing the sum of all values greater than or equal to a value all have a  $\Theta(n)$  complexity.
- Binary search over the [minimum, maximum] interval has complexity of  $O(\log_2(\text{maximum} - \text{minimum}))$ . Assuming that maximum - minimum = range, total complexity is  $O(n * \log_2 \text{range})$ . We can have best case when the first middle already gives us the required elements ( $\Theta(n)$ ) and worst case when we do all the iterations of the binary search ( $\Theta(n * \log_2 \text{range})$ ).

XI. Similar to the idea of doing a few iterations of a sorting algorithm, we can use a part of Quicksort. Quicksort chooses a pivot and rearranges the content of the array in such a way that all elements less than the pivot are in front of it and all elements greater than the pivot are after it. The pivot is on its final position. After this step (let's call it *rearrange*), Quicksort has a recursive call on the elements less than the pivot and a recursive call on elements greater than the pivot. From QuickSort we will use *rearrange* only, in the following way:

- Call *rearrange* on the initial array, choosing an element as pivot (I will choose the last one) and assume that the algorithm, besides rearranging the array, returns also the final position of the pivot, let's call it *pivotPosition*.
- Now (assuming position counting from 1), the array has *pivotPosition* - 1 elements less than the pivot and  $n - \text{pivotPosition}$  elements greater than the pivot. Let's call them *nrLessThan* and *nrGreaterThan*
- We have 4 cases:
  - i.  $k = \text{nrGreaterThan} \Rightarrow$  we need to sum up all the elements from *pivotPosition*+1 to  $n$

- ii.  $k = nrGreater\Than + 1 \Rightarrow$  we need to sum up all the elements from *pivotPosition* to  $n$
- iii.  $k < nrGreater\Than \Rightarrow$  all the elements that we are looking for are after the pivot. We can ignore the elements less than the pivot and have a recursive call to determine the  $k$  largest elements from the array between positions  $pivotPosition+1$  and  $n$ .
- iv.  $k > nrGreater\Than + 1 \Rightarrow$  we need the pivot, all the elements greater than the pivot, plus some elements less than the pivot. So, we first need to sum up all the elements from *pivotPosition* to  $n$ . There are a total of  $nrGreater + 1$  elements here, we still need  $k - (nrGreater + 1)$  elements (let's call this *stillNeeded*). So, we do a recursive call to determine the sum of the *stillNeeded* largest elements from the elements on positions 1 to  $pivotPosition - 1$  and add this to the sum of elements from *pivotPosition* to  $n$ .

○ Complexity:

- i. Rearranging is  $\Theta(n)$ . Finding the sum of elements from *pivotPosition* to  $n$  is  $O(n)$
- ii. we have a  $\Theta(n)$  best case, when one single call to *rearrange* is enough.
- iii. Worst case is when we always pick as pivot the minimum or maximum element and reduce the array to one of size  $n - 1$ . In this case complexity is  $\Theta(n^2)$ , but this is unlikely to happen. Average case, however is  $\Theta(n)$ .